PBL PROJECT · ARCHITECTURE BLUEPRINT

AI-Based Crop Recommendation System

System architecture, research-to-implementation blueprint & development roadmap

PBL PROJECT DOCUMENTATION

Prepared by: Sharad | PRN: 24070521025

Programme: B.Tech Computer Science & Engineering

PBL Guide: Prof. Riddhi Doshi | AY 2025-26

Executive Snapshot

This blueprint turns your research spreadsheet into a concrete build plan. The project decision is to ship **v1 as a Flask web application backed by a trained Random Forest classifier**, using the seven core features every reference paper agrees on: nitrogen (N), phosphorus (P), potassium (K), pH, temperature, humidity and rainfall. Location, weather forecasting, economic ranking, explainability and multilingual support are kept as modular extensions that plug in around this core without a rewrite.

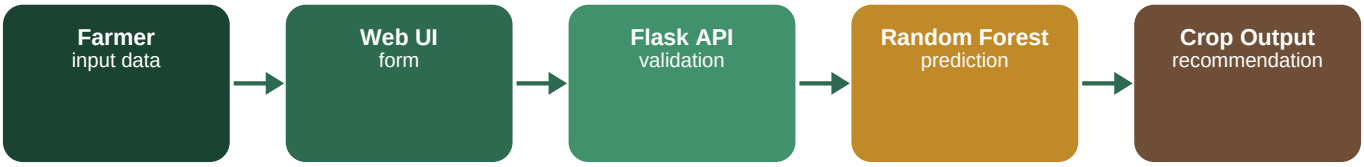

Figure 1. Core request path — v1 scope. Everything else in this document is an optional module layered around this path.

Core inputs

Feature	Meaning	Typical source
N, P, K	Nitrogen, phosphorus, potassium levels in soil	Soil test / lab report
pH	Soil acidity / alkalinity	Soil test
Temperature, Humidity, Rainfall	Local climate conditions	Weather station / API

Key design principle

Random Forest is **the ML component, not the whole product.** The product is the full chain: input capture → validation → preprocessing → prediction → explanation → farmer-facing recommendation.

1 · What Your Research Already Tells You

Your spreadsheet research points to a consistent baseline: crop recommendation is a supervised classification problem built on soil nutrients and environmental variables. The strongest project direction is to turn that baseline into a **farmer-oriented system**, not stop at a notebook prediction.

Research themes

- **Common features** — N, P, K, pH, temperature, humidity and rainfall recur across almost every source.
- **Random Forest** — repeatedly used as a practical nonlinear baseline for structured soil/weather data.
- **Maharashtra relevance** — one paper directly studies crop recommendation and forecasting for Maharashtra, your project's natural regional anchor.
- **Future improvements** — forecast-aware recommendations, economic factors, explainability, multilingual support and farmer Q&A recur as open directions.

Research-to-project mapping

Paper / source	Main contribution	Use in our project
Maharashtra crop recommendation + LSTM (2024)	Regionalisation and future weather prediction	District/location as a planned input and extension point
Soil information + ML (2025)	NPK + pH + environmental variables	This feature family becomes v1's schema
Environmental + economic factors (2025)	Profit / market context	Future ranking layer
Soil/weather/agronomic factors (2025)	RF, DT and SVM baseline comparison	Closest practical reference for v1
Deep learning / multilingual / RAG directions	Forecasting, language and Q&A	Modular interfaces for later phases

2 · Proposed High-Level Architecture

A four-layer architecture keeps concerns cleanly separated: presentation, application, machine-learning service, and data. Each layer only talks to the one directly beneath it, so any layer can be improved independently later.

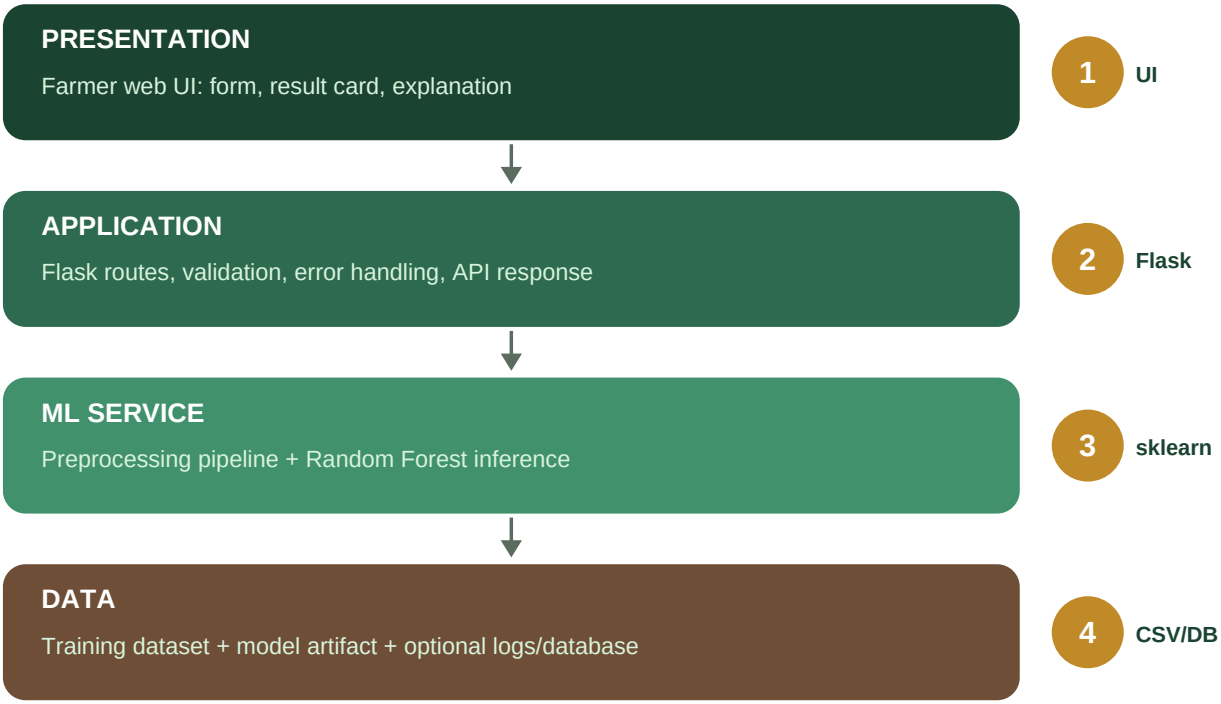

Figure 2. Four-layer architecture for the system.

Optional modules around the core

Plugs into the core

- Weather API — real-time input
- Location — district / agro-climatic context
- Explainability — feature importance / reasons
- Database — history / user logs (optional)

Stays fixed as CORE

- Flask application layer
- Random Forest classifier
- Seven-feature input schema
- Saved scikit-learn Pipeline

Recommended stack

Layer	Choice
Frontend	HTML + CSS + JavaScript
Backend	Python + Flask
ML	scikit-learn RandomForestClassifier, saved as one Pipeline
Data	pandas + NumPy; CSV for v1, database only when needed
Deployment	WSGI server + a cloud host for the final demo

3 · End-to-End User Flow

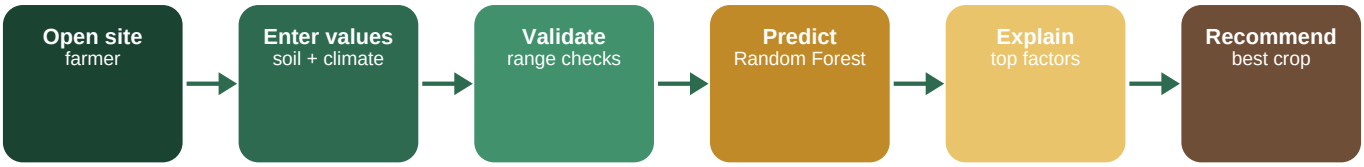

Figure 3. Main request flow, from open site to final recommendation.

Detailed flow

1. **Show the recommendation form**
   Simple entry point for a first-time farmer user
2. **Capture inputs**
   N, P, K, pH, temperature, humidity, rainfall; optionally district/location
3. **Validate**
   Check type, missing values, sensible ranges and units
4. **Preprocess**
   Apply exactly the same transformations and feature order used in training
5. **Call Random Forest**
   Get class prediction and probability estimates
6. **Return top crop(s)**
   Top-1 result, or optionally top-3 with confidence
7. **Generate an explanation**
   Translate model behaviour into plain language
8. **Display the result**
   Crop, confidence, input summary and next-step guidance

Critical rule

Keep the **feature order and preprocessing identical** between training and Flask inference. A silent mismatch here is the single most common bug in student ML web apps.

4 · Machine Learning Pipeline

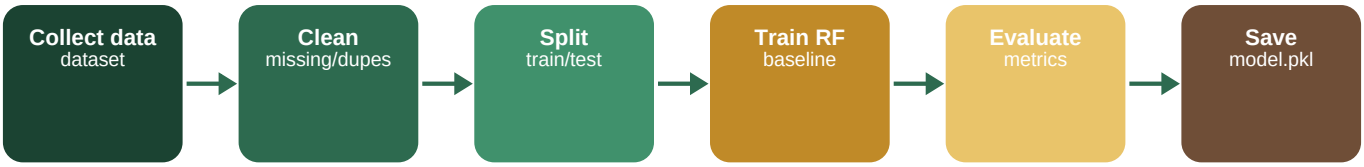

Figure 4. Offline training pipeline — run once, well before the live demo.

Suggested v1 feature schema

Lock this schema and reuse it everywhere

N, P, K, pH, temperature, humidity, rainfall → **7 numeric features.** If your dataset differs, pick one column order on day one and never change it silently.

Training procedure

1. **Inspect dataset**
   Check target classes, missing values, duplicates, class distribution
2. **Prepare X and y**
   X = input columns; y = crop label; preserve a fixed column order
3. **Split the data**
   Stratified hold-out set so every crop class is represented
4. **Train baseline**
   Random Forest first; tune only after a reproducible baseline exists
5. **Evaluate**
   Accuracy, precision, recall, F1, confusion matrix, class-wise behaviour
6. **Serialize pipeline**
   Save preprocessing + model together to prevent deployment mismatch

5 · Website + Backend Design

Recommended screens

Screen	Purpose
Home	Purpose statement + start-recommendation button
Recommendation form	Grouped inputs so the farmer is not overwhelmed
Result card	Recommended crop, confidence, alternatives and explanation
About model	Short explanation, data inputs, limitations and disclaimer
Future advisory page	Forecast, market price, language and Q&A

Suggested API contract

Route	Method → behaviour	Purpose
/	GET → render home	Simple page entry point
/predict	POST → prediction	Validate input, return crop recommendation
/health	GET → health check	Verify server and model availability
/about	GET → model info	Explain project and responsible use

Suggested project structure

```
crop-recommendation/
|-- app.py
|-- model/
|   |-- train.py
|   |-- model_pipeline.pkl
|   `-- metadata.json
|-- data/crop_data.csv
|-- templates/index.html
|-- templates/result.html
|-- templates/about.html
|-- static/css/style.css
|-- static/js/script.js
|-- requirements.txt
`-- README.md
```

6 · Development Roadmap

Build a complete vertical slice before adding advanced modules. Each phase below should be individually demoable before you move to the next.

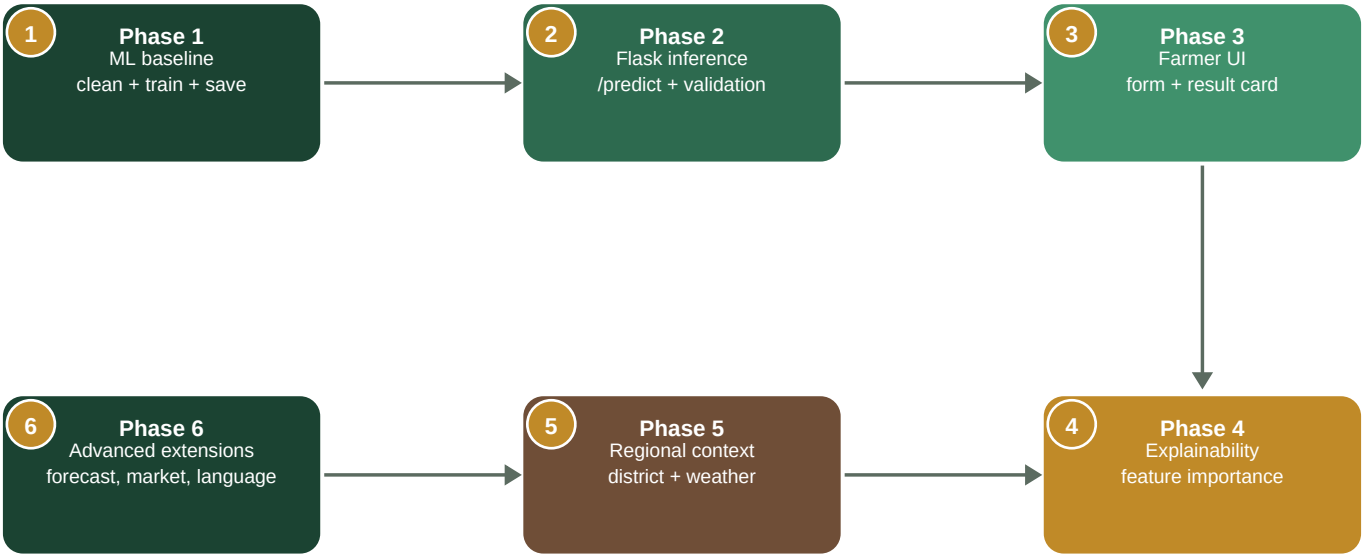

Figure 5. Six-phase build order — a working vertical slice by Phase 3, extensions layered on afterwards.

Recommended build order

Discipline note

Do not start with chatbots, deep learning or multiple external APIs. Prove input → Random Forest → crop → explanation first — everything else is an extension on top of a working core.

7 · Research Gap and Project Novelty

A plain NPK + weather classifier is common in the literature. Your stronger PBL story is to build a **farmer-oriented, extensible system** around the Random Forest baseline, rather than stopping at the classifier itself.

Possible contribution statement

"We propose a modular crop recommendation web system that combines soil and environmental parameters with Random Forest classification, presents farmer-friendly explanations, and is designed for future regional weather forecasting, economic context and multilingual advisory support."

Novelty ladder

1. **Level 1**
   Random Forest prediction from NPK + weather
2. **Level 2**
   Top-3 recommendations with confidence scores
3. **Level 3**
   Model explanation instead of only a crop label
4. **Level 4**
   District-aware and weather-aware recommendations
5. **Level 5**
   Economics, multilingual advisory and AI Q&A

What to measure

- Held-out accuracy and class-wise metrics.
- Prediction latency after the model has loaded.
- Input validation coverage.
- Usability — can a first-time user understand what to enter and why a crop was recommended?
- Robustness on missing or extreme input values.

8 · Testing, Responsibility and Final Demo

Before the final demo, verify the **complete chain**, not just the training notebook.

✓	Model loads correctly from a fresh process.
✓	Website input order matches the model's training schema.
✓	Blank / text / negative / out-of-range values are handled safely.
✓	Valid inputs always return a result.
✓	Result page works on laptop and mobile widths.
✓	Explanation describes model behaviour without claiming causality.
✓	Deployment health route is available and dependencies are pinned.
✓	Report clearly separates implemented features from future scope.

Responsible recommendation note

Present the output as a **model recommendation, not a guarantee** of yield or profit. Real farming decisions also depend on variety, irrigation, disease pressure, local agronomy, season and market conditions.

Suggested final demo story

Figure 6. A compact demo path for the faculty presentation.

9 · Research References From Your Spreadsheet

Use the original publisher or preprint pages for the final bibliography formatting.

1. Crop recommendation and forecasting system for Maharashtra using machine learning with LSTM: a novel expectation-maximization technique (2024)
https://link.springer.com/article/10.1007/s43621-024-00292-5

2. Incorporating soil information with machine learning for crop recommendation to improve agricultural output (2025)
https://doi.org/10.1038/s41598-025-88676-z

3. Crop Recommendation with Machine Learning: Leveraging Environmental and Economic Factors for Optimal Crop Selection (2025)
https://arxiv.org/abs/2505.21201

4. Crop Recommendation Using Machine Learning Based on Soil, Weather, and Other Agronomic Factors (2025)
https://www.ijraset.com/research-paper/crop-recommendation-using-machine-learning-based-on-soil-weather

5. Crop Recommendation System Using Machine Learning (2024)
https://ijarcce.com/papers/crop-recommendation-system-using-machine-learning/

6. Agricultural Recommendation System based on Deep Learning: A Multivariate Weather Forecasting Approach (2024)
https://arxiv.org/abs/2401.11410

7. Crop Recommendation and Agricultural Query Answering System Using Spatio-Temporal Graph Neural Networks and Hybrid Retrieval Augmentation (2026)
https://arxiv.org/abs/2606.09160

8. Deep Learning-Based Multilingual Smart Farming System for Crop Recommendation and Nutrient Monitoring (2026)
https://www.ajaronline.com/index.php/AJAR/article/view/1898

Closing note

This blueprint is a living plan, not a locked spec. Revisit the novelty ladder each milestone and update this document as decisions firm up ahead of the PBL review.